

Q.No:9

Problem Description:

You are given a rooted tree that contains N nodes. Each node contains a lowercase alphabet.

You are required to answer Q queries of type u, c , where u is an integer and c is a lowercase alphabet. The count of nodes in the subtree of the node u containing c is considered as the answer of all the queries.

Constraints

$$1 \leq N, Q \leq 10^5$$

$$1 \leq u, v \leq N$$

- c is a lowercase alphabet
- s_i is a lowercase alphabet for all $1 \leq i \leq N$
- 1 is the root node

Input format

- First line: Two space-separated integers N and Q respectively
- Second line: A string s of length N (where the i^{th} character of s represents the character stored in node i)
- Next $N - 1$ line: Two space-separated integers u and v denoting an edge between node u and node v
- Next Q lines: An integer u and a space-separated character c

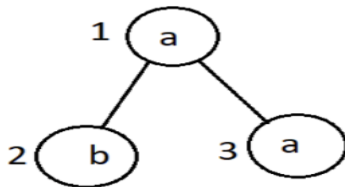
Output format

For each query, print the output in a new line.

Note: It is guaranteed that the input generates a valid tree.

Explanation for test case 1

Tree given in the sample input will look like that.



Number of nodes in the subtree of node 1 having 'a' stored in it is 2.

Logical Test Cases

Test Case 1

INPUT (STDIN)

```
3 1
aba
1 2
1 3
1 a
```

EXPECTED OUTPUT

```
2
```

Test Case 2

INPUT (STDIN)

```
4 1
WSX
1 2
1 3
1 1
1 W
```

EXPECTED OUTPUT

```
10
```

Mandatory Test Cases

Test Case 1

KEYWORD

```
while(Q--)
```

Test Case 2

KEYWORD

```
for(i = 0; i < N - 1; i++)
```

Code:

```
#include <stdio.h>
#include <stdlib.h>

int main()
{
    int N, Q;
    scanf("%d %d", &N, &Q);

    char *s = (char *)malloc((N + 1) * sizeof(char));
    scanf("%s", s);

    int *head = (int *)malloc(N * sizeof(int));
    int *to = (int *)malloc(2 * (N - 1) * sizeof(int));
```

```
int *next = (int *)malloc(2 * (N - 1) * sizeof(int));
int *parent = (int *)malloc(N * sizeof(int));
int *order = (int *)malloc(N * sizeof(int));
int *stack = (int *)malloc(N * sizeof(int));
int *cnt = (int *)calloc(N * 26, sizeof(int));

int i;

for (i = 0; i < N; i++)
    head[i] = -1;

int edge = 0;

for (i = 0; i < N - 1; i++)
{
    int u, v;
    scanf("%d %d", &u, &v);
    u--;
    v--;

    to[edge] = v;
    next[edge] = head[u];
    head[u] = edge++;

    to[edge] = u;
    next[edge] = head[v];
    head[v] = edge++;
}

int top = 0;
int ord = 0;

stack[top++] = 0;
parent[0] = -1;

while (top > 0)
{
    int u = stack[--top];
    order[ord++] = u;

    for (i = head[u]; i != -1; i = next[i])
    {
        int v = to[i];

        if (v == parent[u])
            continue;

        parent[v] = u;
        stack[top++] = v;
    }
}
```

```
for (i = 0; i < N; i++)
    cnt[i * 26 + (s[i] - 'a')] = 1;

for (i = N - 1; i > 0; i--)
{
    int u = order[i];
    int p = parent[u];

    int j;
    for (j = 0; j < 26; j++)
        cnt[p * 26 + j] += cnt[u * 26 + j];
}

while (Q--)
{
    int u;
    char c;

    scanf("%d %c", &u, &c);

    u--;

    printf("%d\n", cnt[u * 26 + (c - 'a')]);
}

free(s);
free(head);
free(to);
free(next);
free(parent);
free(order);
free(stack);
free(cnt);

return 0;
}
```

Output:

```
Test_case:1
aadityajaiswal@Aadityas:~$ ./a.out
3 1
aba
1 2
1 3
1 a
2
aadityajaiswal@Aadityas:~$
```

Test_case:2

```
● aadityajaisw  
4 1  
wsx  
1 2  
1 3  
1 1  
1 w  
2  
● aadityajaisw
```